

HDOC DAY – 28

1) 2D array: column-wise input and row-major output using a single loop

ALGORITHM:

1. Start.
2. Input the number of rows.
3. If $\text{rows} \leq 0$, display an error and ask again.
4. Input the number of columns.
5. If $\text{columns} \leq 0$, display an error and ask again.
6. Declare a variable-length 2D array `a[rows][cols]`.
7. Read elements column by column:
 - Outer loop runs through columns.
 - Inner loop runs through rows.
8. Use one loop from 0 to $\text{rows} * \text{cols} - 1$.
9. For each linear index `i`:
 - Row index = i / cols
 - Column index = $i \% \text{cols}$
10. Print the element `a[i / cols][i % cols]`.
11. Stop.

C PROGRAM:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int rows, cols, i, j;
```

```
    /* Read and validate number of rows */
```

```
    do
```

```
{  
    printf("Enter number of rows: ");  
    scanf("%d", &rows);  
  
    if (rows <= 0)  
        printf("Rows must be positive. Enter again.\n");  
  
} while (rows <= 0);  
  
/* Read and validate number of columns */  
do  
{  
    printf("Enter number of columns: ");  
    scanf("%d", &cols);  
  
    if (cols <= 0)  
        printf("Columns must be positive. Enter again.\n");  
  
} while (cols <= 0);  
  
/* Variable Length Array */  
int a[rows][cols];  
  
/* Reading elements column by column */  
printf("\nEnter the array elements column by column:\n");  
  
for (j = 0; j < cols; j++)  
{
```

```

    for (i = 0; i < rows; i++)
    {
        printf("a[%d][%d] = ", i, j);

        scanf("%d", &a[i][j]);
    }
}

/* Printing in row-major order using a single loop */
printf("\nArray in row-major order:\n");

for (i = 0; i < rows * cols; i++)
{
    printf("%d ", a[i / cols][i % cols]);

    /* Move to next line after each row */
    if ((i + 1) % cols == 0)
        printf("\n");
}

return 0;
}

```

TEST CASE TABLE:

Test case table	rows	columns	Elements entered column wise	Expected row-major output
1	2	3	1 2 3 4 5 6	1 3 5 / 2 4 6
2	2	2	10 20 30 40	10 30 / 20 40
3	1	3	5 10 15	5 10 15
4	0 then 2	2	1 2 3 4	Error for 0, then accept 2

5	2	-1 then 2	1 2 3 4	Error for -1 then accept 2
---	---	-----------	---------	-------------------------------

```

./h28
Enter number of rows: 2
Enter number of columns: 3

Enter the array elements column by column:
a[0][0] = 1
a[1][0] = 2
a[0][1] = 3
a[1][1] = 4
a[0][2] = 5
a[1][2] = 6

Array in row-major order:
1 3 5
2 4 6

```

2) Copy a variable-length 2D array into a 1D array in row-major order

Here the input is taken normally into the 2D array. We then copy all elements into a 1D array in row-major order.

ALGORITHM:

1. Start.
2. Input rows and columns.
3. Validate that both are positive.
4. Declare the VLA `a[rows][cols]`.
5. Declare the 1D VLA `b[rows * cols]`.
6. Read elements into the 2D array using nested loops.
7. Set `k = 0`.
8. Traverse the 2D array row by row.
9. Copy `a[i][j]` into `b[k]`.
10. Increment `k`.
11. Display the 2D array.
12. Display the 1D array.
13. Stop.

C PROGRAM:

```
#include <stdio.h>
```

```
int main()
{
    int rows, cols, i, j, k = 0;

    /* Input and validation of rows */
    do
    {
        printf("Enter number of rows: ");
        scanf("%d", &rows);

        if (rows <= 0)
            printf("Rows must be positive. Enter again.\n");

    } while (rows <= 0);

    /* Input and validation of columns */
    do
    {
        printf("Enter number of columns: ");
        scanf("%d", &cols);

        if (cols <= 0)
            printf("Columns must be positive. Enter again.\n");

    } while (cols <= 0);

    /* Variable Length Arrays */
    int a[rows][cols];
```

```
int b[rows * cols];
```

```
/* Reading the 2D array */
```

```
printf("\nEnter the elements of the 2D array:\n");
```

```
for (i = 0; i < rows; i++)
```

```
{
```

```
    for (j = 0; j < cols; j++)
```

```
    {
```

```
        printf("a[%d][%d] = ", i, j);
```

```
        scanf("%d", &a[i][j]);
```

```
    }
```

```
}
```

```
/* Copying 2D array into 1D array in row-major order */
```

```
for (i = 0; i < rows; i++)
```

```
{
```

```
    for (j = 0; j < cols; j++)
```

```
    {
```

```
        b[k] = a[i][j];
```

```
        k++;
```

```
    }
```

```
}
```

```
/* Display the 2D array */
```

```
printf("\n2D Array:\n");
```

```
for (i = 0; i < rows; i++)
```

```

{
    for (j = 0; j < cols; j++)
    {
        printf("%d ", a[i][j]);
    }
    printf("\n");
}

/* Display the 1D array */
printf("\n1D Array in row-major order:\n");

for (i = 0; i < rows * cols; i++)
{
    printf("%d ", b[i]);
}

printf("\n");

return 0;
}

```

TEST CASE TABLE:

Test case	rows	columns	2D array	Expected 1D array
1	2	3	1 2 3 / 4 5 6	1 2 3 4 5 6
2	3	2	1 2 / 3 4 / 5 6	1 2 3 4 5 6
3	1	4	10 20 30 40	10 20 30 40
4	2	2	5 10 / 15 20	5 10 15 20
5	-2 then 2	2	1 2 / 3 4	Error first, then 1 2 3 4